

MODULE 08 · PART 2 - WINDOW FUNCTIONS

Filtering on a Window Result

QUALIFY where it exists, the wrapper where it does not, and how to write code that runs on both.

PART 2 of 8 MODULE 8 of 26 RUNS about 24 min

Why this one is short

- ◆ **There are only two answers**, and one of them is unavailable on half the engines in common use.
- ◆ **The syntax takes five minutes.** QUALIFY is one keyword, and the wrapper is a shape you already write.
- ◆ **The decision underneath is not short.** Filtering before or after the window produces two different reports.
- ◆ **So most of this module is that decision**, and only a few slides are about the clause itself.

THE TAKEAWAY

The syntax is trivial and the placement is not. Every scenario in this module is about which side of step five a condition landed on.

Window results exist at step five. Every clause that filters runs **either side of it**, never at it.

The three legal places, again

WHERE YOU FILTER	WHAT IT MEANS
QUALIFY	Step six. Immediately after the window, in the same query block.
An outer query block	The window result becomes an ordinary column, filtered in the outer WHERE.
ORDER BY with LIMIT	Step nine and ten. The top of the whole result, not per group.
WHERE, GROUP BY, HAVING	Not available. All three run before step five.

THE TAKEAWAY

Only the first two are real choices. The third answers a different question and the fourth is the error everybody writes first.

QUALIFY is to window functions what
HAVING is to aggregates. **The same idea,
one step later.**

The parallel, exactly

HAVING

Filters on an aggregate

```
1 SELECT store_key,  
2       SUM(net_amount) AS sales  
3 FROM fact_sales_line  
4 GROUP BY store_key  
5 HAVING SUM(net_amount) > 100000;
```

Step four. The aggregate was formed at step three, one step earlier.

QUALIFY

Filters on a window

```
1 SELECT store_key, product_key, net_amount  
2 FROM fact_sales_line  
3 QUALIFY ROW_NUMBER() OVER (  
4         PARTITION BY store_key  
5         ORDER BY net_amount DESC, sale_key  
6         ) <= 3;
```

Step six. The window was computed at step five, one step earlier.

THE TIEBREAKER

Two clauses, one idea. Each sits immediately after the step that creates the thing it filters on, because there is nowhere else it could sit.

What QUALIFY can see

- ◆ **Window function results**, written out in full, or referenced by their SELECT alias on engines that allow it.
- ◆ **Aggregates and grouping keys**, because HAVING ran two steps earlier and those already exist.
- ◆ **Ordinary columns**, so a condition can mix a rank with a plain predicate in one expression.
- ◆ **It must contain at least one window function**. A QUALIFY with only ordinary predicates is rejected, and it belongs in WHERE anyway.

THE TAKEAWAY

QUALIFY sits later than every other filtering clause, so it can see everything they can see and one thing they cannot.

QUALIFY, by engine

ENGINE	QUALIFY
Teradata	Yes. The clause originates here, decades before the others.
Snowflake	Yes, and it can reference SELECT aliases.
Amazon Redshift	Yes, since July 2023.
Databricks, BigQuery, DuckDB	Yes.
PostgreSQL, SQL Server, MySQL, Oracle	No. Wrap the query instead.

THE TAKEAWAY

Not in the SQL standard, and not likely to be. It is a vendor extension that four warehouse vendors adopted independently because the need is universal.

The three things QUALIFY is for

```
1 -- 1. deduplicate
2 QUALIFY ROW_NUMBER() OVER (
3   PARTITION BY customer_durable_key
4   ORDER BY valid_from DESC, customer_key DESC) = 1
5
6 -- 2. top N per group
7 QUALIFY RANK() OVER (
8   PARTITION BY region_key ORDER BY sales DESC) <= 3
9
10 -- 3. rows that changed
11 QUALIFY price IS DISTINCT FROM LAG(price) OVER (
12   PARTITION BY product_key ORDER BY price_date)
```

11 The third one is the least known and the most useful. Keep only the rows where something actually moved.

THE TAKEAWAY

Deduplicate, top N, and change detection. Almost everything QUALIFY is used for is one of these three, and the third is why it beats a GROUP BY.

Without QUALIFY, you promote the window result to **an ordinary column** and filter it outside.

One query, two dialects

QUALIFY

Redshift, Snowflake, DuckDB

```

1 SELECT store_key, product_key, net_amount
2   FROM fact_sales_line
3  QUALIFY ROW_NUMBER() OVER (
4          PARTITION BY store_key
5          ORDER BY net_amount DESC, sale_key
6          ) <= 3;

```

Six lines, one block, and no window column in the output.

CTE

Postgres, SQL Server, anywhere

```

1 WITH ranked AS (
2   SELECT store_key, product_key, net_amount,
3          ROW_NUMBER() OVER (
4          PARTITION BY store_key
5          ORDER BY net_amount DESC, sale_key
6          ) AS rn
7   FROM fact_sales_line
8 )
9  SELECT store_key, product_key, net_amount
10 FROM ranked WHERE rn <= 3;

```

Ten lines, two blocks, and line 9 has to list the columns to keep rn out.

THE TIEBREAKER

Identical meaning and, on every engine worth using, an identical plan. The difference is four lines and one opportunity to leak a column.

The column that leaks

- ◆ **SELECT *** inside the CTE projects the rank column, and SELECT * outside it carries that column into the result.
- ◆ **Nothing errors.** The consumer simply receives one more column than it did yesterday, in the middle of the list.
- ◆ **Positional consumers break.** Anything reading by index rather than by name silently shifts by one.
- ◆ **List the columns in the outer SELECT.** It is the only difference between the two forms that can reach production.

THE TAKEAWAY

QUALIFY cannot leak a column, because the window is never projected. That is a genuine advantage and it is not the one people cite.

Subquery, or common table expression

```
1 -- as a CTE
2 WITH ranked AS (
3     SELECT store_key, net_amount,
4           ROW_NUMBER() OVER (...) AS rn
5     FROM fact_sales_line)
6 SELECT store_key, net_amount FROM ranked WHERE rn = 1;
7
8 -- as an inline subquery
9 SELECT store_key, net_amount
10 FROM (SELECT store_key, net_amount,
11           ROW_NUMBER() OVER (...) AS rn
12        FROM fact_sales_line) t
13 WHERE rn = 1;
```

10 Identical meaning and, on every modern engine, an identical plan. The CTE is named and the subquery is not.

THE TAKEAWAY

Prefer the CTE. It reads top to bottom, the name documents what the block is, and it can be selected from on its own while debugging.

Why analysts write CTEs

- ◆ **They read in execution order**, roughly, which an inline subquery does not. You read the inner one last and it ran first.
- ◆ **They can be stacked**. Each level names one idea, and the next level treats it as an ordinary table.
- ◆ **They can be selected from during development**, by commenting out the final SELECT, which subqueries cannot.
- ◆ **The performance folklore about them is stale**, and module ten is entirely about why.

THE TAKEAWAY

The readability argument is real and it is not the whole story. Module ten covers when a CTE is inlined, when it is materialised, and when that distinction matters.

A customer deduplication written with QUALIFY on the Redshift warehouse is ported to the Postgres store-ops replica as a CTE. Both the inner and outer SELECT use a star. The ported query returns the same rows and the same row count.

Two days later a downstream extract feeding a partner file starts failing validation. The partner reports that the postcode field now contains integers. Nothing about the customer data has changed.

What happened?

- ☐ A The CTE applies the filter at a different step, so the row count changed and the file is misaligned.
- ☐ B Postgres returns columns in a different order from Redshift, so the partner's positional mapping broke.
- ☐ C ROW_NUMBER is non-deterministic across engines, so a different row survived for some customers.
- ☐ D SELECT * projected the rank column through both blocks, shifting every field after it by one position.

◆ PAUSE AND DECIDE

What happened?

A Filter at a different step

The two forms filter at the same point and the scenario says the row count is unchanged. Rows are not the problem here.

C A different row survived

The ordering ends in `customer_key`, so it is total and the survivor is the same on both engines. Module two was handled correctly.

B Different column order

Plausible on a star and not what happened. Both engines return the base columns in table order, and one extra column was added.

D The rank column leaked

An extra integer column appears where the star expanded it, and every field after it moves one place to the right.

THE SIGNAL

"The postcode field now contains integers." A field holding the wrong type, with the right row count, is a column-position problem and almost never a data one.

QUALIFY is **not faster**. It is the same plan, written in four fewer lines.

Where the cost actually is

- ◆ **The sort.** Both forms compute the same window over the same rows, and the sort is the expensive part of that.
- ◆ **Neither form can filter before the window.** Step six and the outer WHERE are both after step five, by definition.
- ◆ **So push predicates into the inner block** wherever the meaning allows. A filter at step two shrinks what step five has to sort.
- ◆ **And check the meaning when you do,** because filtering before the window changes every window result. Module three.

THE TAKEAWAY

The only real optimisation here is moving work before the window rather than after it, and that is a change of meaning as often as it is a change of speed.

A team on Redshift rewrites forty reporting queries from the CTE wrapper form to QUALIFY, on the basis that QUALIFY avoids materialising an intermediate result and will therefore be faster.

After the rewrite, measured runtimes are unchanged to within noise on all forty. The lead asks whether the migration was worth doing and whether something was configured wrongly.

What should they be told?

- ☐ A The CTEs needed a materialisation hint removed as part of the rewrite, which was missed.
- ☐ B QUALIFY is faster only on tables with a matching distribution key, which these are not.
- ☐ C QUALIFY compiles to the same plan. The gain is readability, and the cost is the sort, which neither form avoids.
- ☐ D The queries also need a LIMIT so that the engine can short-circuit the window computation.

◆ PAUSE AND DECIDE

What should they be told?

A A missed materialisation hint

Assumes the CTE was materialised. A single-reference CTE is inlined on modern engines, which is module ten, so there was nothing to remove.

C Same plan, readability gain

Forty queries got shorter and one leak class disappeared.
Unchanged runtime is the expected result, not a symptom.

B Distribution key dependent

Invents a rule. Distribution affects how the window is computed and it affects both forms identically, because they are the same plan.

D Add a LIMIT

A LIMIT can help a global top-N and does nothing for a per-group one, because every partition still has to be ranked.

THE SIGNAL

"Unchanged to within noise on all forty." A rewrite that changes nothing measurable, on every query, is a rewrite that changed the text and not the plan.

Top of everything, or top of each group

```
1 -- top 10 overall: no window needed at all
2 SELECT store_key, product_key, net_amount
3   FROM fact_sales_line
4  ORDER BY net_amount DESC, sale_key
5  LIMIT 10;
6
7 -- top 3 per store: a window is required
8 SELECT store_key, product_key, net_amount
9   FROM fact_sales_line
10 QUALIFY ROW_NUMBER() OVER (PARTITION BY store_key
11                            ORDER BY net_amount DESC, sale_key) <= 3;
```

- 5 Ten rows in total. The engine can often stop early, and no window is computed at all.
- 11 Three rows per store, so 1,236 rows. Every partition has to be ranked before anything can be returned.

THE TAKEAWAY

These look like the same request in English and they are completely different queries. Only one of them needs a window function.

When you do not need a window at all

- ◆ **Top N of the whole result is ORDER BY and LIMIT.** Cheaper, simpler, and often short-circuited by the engine.
- ◆ **One row per group with only a few columns is GROUP BY with MIN or MAX,** which needs no ranking and no filter.
- ◆ **Reach for a window when you need whole rows, or per-group positions,** and not before.
- ◆ **The most impressive tool is not the default one.** Module four's table said the same thing about deduplication.

THE TAKEAWAY

A window function filtered by QUALIFY is the right answer to a narrower set of questions than the number of times it gets written.

Moving a condition from HAVING to QUALIFY **changes the answer**, because the window sees different rows.

The same condition, two positions

```
1 SELECT s.region_key, f.store_key,  
2        SUM(f.net_amount) AS sales  
3 FROM fact_sales_line f  
4 JOIN dim_store s USING (store_key)  
5 GROUP BY s.region_key, f.store_key  
6 HAVING SUM(f.net_amount) > 100000  
7 QUALIFY RANK() OVER (PARTITION BY s.region_key  
8                     ORDER BY SUM(f.net_amount) DESC) <= 3;
```

- 6 Removes small stores before ranking, so the ranks describe only stores above the threshold.
- 7 Move that threshold down here and the ranks describe every store. Same rows kept, different rank values.

THE TAKEAWAY

Both clauses in one query is legal and common. Which side of the window each condition sits on is a decision about what the rank means.

Which side should a condition go

- ◆ **Before the window, if the excluded rows are not part of the comparison.** Test stores should not be ranked at all.
- ◆ **After the window, if the excluded rows are part of the population** but not part of the answer. Rank everyone, show three.
- ◆ **Ask what the rank claims.** 'Third of all stores' and 'third of qualifying stores' are different sentences.
- ◆ **There is no default.** Whichever clause you happened to write in is the answer you published.

THE TAKEAWAY

This is the same question as module three's 'rank within what', arriving as a choice between two clauses rather than as a partition.

Rows that changed, and nothing else

```
1 SELECT product_key, price_date, price,  
2         LAG(price) OVER w AS prev_price  
3     FROM vw_price_spine  
4     WINDOW w AS (PARTITION BY product_key ORDER BY price_date)  
5     QUALIFY price IS DISTINCT FROM LAG(price) OVER w;
```

5 IS DISTINCT FROM, so the first row of each product qualifies. With <> it would be dropped, silently.

THE TAKEAWAY

A change log built from a full snapshot, in five lines. Every price change, and no rows at all on the days nothing moved.

More than one condition in a QUALIFY

```
1 SELECT customer_key, sold_ts, net_amount
2   FROM fact_sales_line
3  WHERE date_key >= 20260101
4  QUALIFY ROW_NUMBER() OVER w <= 5
5         AND SUM(net_amount) OVER w > 200
6         AND net_amount > 10
7 WINDOW w AS (PARTITION BY customer_key
8              ORDER BY sold_ts DESC, sale_key)
```

6 A plain predicate inside QUALIFY. Legal, and it would filter more cheaply in the WHERE clause at step two.

THE TAKEAWAY

Line 6 runs at step six on rows that survived the window. Moved to WHERE it runs at step two, and it changes what the other two conditions see.

Two windows in one condition

- ◆ **Conditions can mix different window specifications**, so a rank within store and a share of region can be tested together.
- ◆ **Each distinct specification costs a sort**, so a compound QUALIFY can be more expensive than it looks on the page.
- ◆ **The named WINDOW clause makes the reuse visible**, and makes it obvious when two conditions are not sharing one.
- ◆ **Order the conditions cheapest first where you can**. Most engines short-circuit, and a plain predicate is free by comparison.

THE TAKEAWAY

A QUALIFY holding three different window specifications is three sorts wearing one clause. Nothing in the syntax makes that visible.

Filtering on a **running total** is where this gets interesting, and where people get it wrong.

The Pareto filter, written twice

```
1 -- the obvious version: drops the crossing row
2 QUALIFY SUM(sales) OVER (ORDER BY sales DESC, customer_key
3     ROWS UNBOUNDED PRECEDING)
4     / SUM(sales) OVER () <= 0.80
5
6 -- the correct version: tests the previous row's total
7 QUALIFY COALESCE(SUM(sales) OVER (ORDER BY sales DESC, customer_key
8     ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING), 0)
9     / SUM(sales) OVER () < 0.80
```

8 The frame ends one row early, so the test is on the total before this row. The row that crosses 80% is kept.

THE TAKEAWAY

The first version keeps customers until the total would exceed 80%, so it stops short. The second keeps them until it has, so it covers the threshold.

Why it always errs the same way

- ◆ **The condition excludes the row that satisfies it.** The first customer to push the total past 80% is the one you need.
- ◆ **So the answer is always short, never over.** A one-directional error looks like a definitional difference rather than a defect.
- ◆ **The size of the shortfall depends on the largest excluded customer,** so it varies by period and never settles.
- ◆ **Frames ending at 1 PRECEDING are the general fix,** for any 'until it reaches' threshold, not just Pareto.

THE TAKEAWAY

Every threshold question has this shape. Until the total reaches X means including the row that gets you there, and the obvious condition excludes it.

A commercial team reports the set of customers who together account for 80% of Kestrel's annual sales. The query computes a cumulative share over customers ordered by spend descending, and keeps rows where that share is at or below 0.80.

Finance checks the output and finds the retained customers account for 78.4% of sales, not 80%. Rerun for the previous year the figure is 79.1%. It is always below and never above, and the gap changes each time.

What is the defect?

- ☐ A The condition excludes the row that crosses the threshold. Test the cumulative total up to the previous row instead.
- ☐ B The running total uses RANGE rather than ROWS, so customers with equal spend share a cumulative value.
- ☐ C The share should be recomputed after filtering, so that the retained customers re-normalise to 100%.
- ☐ D NTILE(5) with the first bucket kept is the standard way to express a Pareto cut and avoids the issue.

◆ PAUSE AND DECIDE

What is the defect?

- A

 The crossing row is excluded
The customer who takes the total past 80% fails a test asking whether it is at or below 80%. A frame ending at 1 PRECEDING keeps them.
- B

 RANGE instead of ROWS
A real hazard from module five, and it would make tied customers share a value. Spend to the penny rarely ties, and it would not be one-directional.
- C

 Re-normalise after filtering
Makes the retained set sum to 100% by definition, which hides the question rather than answering it. The 80% claim becomes unfalsifiable.
- D

 Use NTILE(5)
Gives the top fifth of customers by count, which is a different question. Pareto is about share of value, not share of customers.

THE SIGNAL

"Always below and never above." A one-directional error with a varying size is an off-by-one at a boundary, not a calculation error in the body.

Kestrel writes QUALIFY in the warehouse and wrappers in the replica. **Both, deliberately.**

Portable everywhere, or native to each

ONE FORM

Wrappers everywhere

- One version to review
- Runs on both platforms
- Longer, on every query
- Carries the leak risk forever

TWO FORMS

Native to each engine

- Shorter where it can be
- Two versions to keep aligned
- They will drift
- Divergence is silent

THE TIEBREAKER

The real answer is usually neither. Model the logic once as a table in the warehouse, and let the replica read the result rather than recompute it.

If you must write it twice

- ◆ **Generate both from one definition**, with a macro or a templating layer, so there is one source of truth for the logic.
- ◆ **Test both against the same fixture**, and assert the two results are identical in both directions. Module twenty.
- ◆ **Never hand-port and hand-maintain**. Scenario one is what hand-porting produces, and it took two days to surface.
- ◆ **Record which is authoritative**. When they disagree, somebody needs to know which one is the answer.

THE TAKEAWAY

Two implementations of one rule is a reconciliation problem in waiting. Deciding in advance which one is authoritative is the cheapest part of the whole exercise.

An analyst on Redshift needs the top three stores per region, restricted to stores that took more than \$100,000. They write the threshold in HAVING and the rank in QUALIFY, and it works.

A reviewer suggests moving the threshold into QUALIFY as well, so that both conditions sit in one clause and the query reads more consistently. The analyst is unsure whether that is equivalent.

What is the right answer?

- ☐ A Combine both conditions in HAVING, using a subquery to make the rank available at that step.
- ☐ B Keep the threshold in HAVING and the rank in QUALIFY. Moving it changes which stores are ranked, and so changes the ranks.
- ☐ C Move it into QUALIFY. That clause can see aggregates, so the two conditions are equivalent wherever they sit.
- ☐ D Wrap the whole query in a CTE and apply both conditions in the outer WHERE, which is clearer than either clause.

◆ PAUSE AND DECIDE

What is the right answer?

A Combine in HAVING

A rank cannot be made available at step four by any means. The subquery would compute it later, which is the form we already have.

C Move it into QUALIFY

QUALIFY can see aggregates, so it runs. Small stores are then ranked and discarded, so a region's third-placed store may be one that never clears the threshold.

B Leave them where they are

HAVING removes small stores before step five, so the ranks describe qualifying stores. Both placements are defensible and they are not the same report.

D Wrap it and filter outside

Equivalent to option C, in more lines. Both conditions land after the window, which is the placement change nobody intended to make.

THE SIGNAL

"Reads more consistently." A refactor motivated by style, across a boundary that carries meaning. The query still runs, and it answers a different question.

Reading a QUALIFY out loud

WRITTEN

SAID

```
QUALIFY ROW_NUMBER() OVER (...) = 1
```

keep one row per partition, the first by that ordering

```
QUALIFY RANK() OVER (...) <= 3
```

keep the top three, and anything tied with third

```
QUALIFY COUNT(*) OVER (PARTITION BY k) > 1
```

keep only rows whose key appears more than once

```
QUALIFY x IS DISTINCT FROM LAG(x) OVER (...)
```

keep only rows where x changed

THE TAKEAWAY

Row three is a useful one to know. It finds duplicates rather than removing them, which is the query you run before deciding how to deduplicate.

WHERE, HAVING or QUALIFY

THE CONDITION MENTIONS	THE CLAUSE
Only table columns	WHERE. Step two, and it shrinks everything downstream.
An aggregate	HAVING. Step four, after grouping.
A window result	QUALIFY, or an outer query block. Step six or later.
An aggregate, and you want it before ranking	HAVING. The window then sees only qualifying rows.
An aggregate, and you want it after ranking	QUALIFY. The window sees everything, then you filter.

THE TAKEAWAY

The last two rows are the same condition in two clauses, producing two different reports. Nothing in the syntax hints that a choice was made.

Deciding, in three questions

01

What

Does the condition mention a window result? If not, it belongs earlier and this module does not apply.

02

Where

Should the excluded rows have been part of the comparison? That picks before or after the window.

03

How

QUALIFY if the engine has it. An outer block if not, with the columns listed.

THE TAKEAWAY

Step two is the only one with a wrong answer that runs. Steps one and three produce errors or verbosity, which are both cheap.

Testing a filter placement

- ◆ **Run it both ways and diff the results.** If the rows are identical, the placement does not matter on today's data.
- ◆ **Then check the rank values, not only the rows.** The same three stores with different rank numbers is still a difference.
- ◆ **Identical today does not mean identical tomorrow.** The placements diverge when a row moves across the threshold.
- ◆ **Write the intent in a comment.** 'Rank across all stores, then filter' is one line and it survives the next refactor.

THE TAKEAWAY

Scenario four is somebody refactoring across this boundary for style. A comment saying which side was chosen deliberately is what stops that.

Four habits

- ◆ **List the columns in the outer SELECT**, every time you use the wrapper form. Two stars is how a rank column reaches a partner file.
- ◆ **Say what the rank claims** before choosing HAVING or QUALIFY. Third of all, or third of those that qualify.
- ◆ **Use a frame ending at 1 PRECEDING for any 'until it reaches' threshold**, so the crossing row is kept.
- ◆ **Do not claim QUALIFY is faster**. It is shorter and safer, and those are better arguments than one that is untrue.

THE TAKEAWAY

One of these prevents a wrong number, one prevents a broken file, one prevents a mislabelled report, and one prevents an argument you would lose.

Part 2 has one module left. **Module nine** is distributions, and then part three begins.